

Für ein besseres Verständnis und eine bessere Struktur der Antwort sind die ursprünglichen Punkte der Aufgabe **blau markiert**.

Du arbeitest als DevOps-Engineer in einem mittelständischen Unternehmen, das eine neue Produktlinie auf den Markt bringen möchte. Deine Aufgabe ist es, eine robuste CI/CD-Pipeline zu entwerfen und zu implementieren, die die Entwicklung und Bereitstellung der neuen Softwareprodukte unterstützt. Das Unternehmen setzt auf eine Microservices-Architektur, die in Containern auf einer Kubernetes-Cluster-Umgebung gehostet wird. Du sollst folgende Schritte durchführen:

Für diese Aufgabe verwende ich eine kleine Testanwendung, die bereits im Rahmen früherer praktischer Aufgaben vorbereitet wurde. Die Anwendung besteht aus drei Hauptkomponenten: einem nginx-Frontend, einem Python-Backend und einer PostgreSQL-Datenbank.

Die Anwendung ist keine vollständige Produktivumgebung. Sie eignet sich aber gut, um den Weg von einer Codeänderung bis zur Vorbereitung eines Deployments in Kubernetes zu zeigen. Sie wurde in einer lokalen Kubernetes-Laborumgebung getestet. Der Kubernetes-Cluster selbst wird durch dieses Repository nicht automatisch erstellt. Für die Anwendung der Kubernetes-Manifeste wird deshalb ein bereits vorbereiteter Cluster benötigt. In einer Produktivumgebung könnte ein solcher Cluster zum Beispiel automatisiert mit Terraform oder einem vergleichbaren Infrastructure-as-Code-Werkzeug bereitgestellt werden.

Das Repository enthält den Anwendungscode, Dockerfiles, Docker Compose für den lokalen Start, Kubernetes-Manifeste, einen GitHub-Actions-Workflow sowie ein vereinfachtes Terraform-Beispiel für Infrastructure as Code. Es ist hier verfügbar:

<https://github.com/cyber-antony/DevOps-TP1>

A. Entwirf eine CI/CD-Pipeline, die automatische Builds, Tests und Deployments für die Microservices umfasst. Beschreibe, welche Tools du für die CI/CD-Pipeline auswählen würdest und warum. Berücksichtige dabei die Integration von Code-Qualitätsprüfungen und Sicherheitsscans.

Für CI/CD verwenden wir GitHub Actions, weil der Projektcode bereits in GitHub liegt und der Workflow direkt im Repository als Pipeline as Code gespeichert werden kann. Dadurch ist die Pipeline versionierbar, überprüfbar und wiederholbar. Außerdem kann der Workflow automatisch bei einem push in den Branch main oder bei einem pull request gestartet werden.

Die Pipeline ist in der Standarddatei beschrieben: `.github/workflows/ci.yml`

```
name: CI/CD

on:
  push:
    branches:
      - main          # automatischer Start bei Push auf main
  pull_request:      # Start bei Pull Requests
  workflow_dispatch:
    inputs:
      deploy_lab:
        description: "Deploy to local lab Kubernetes cluster"
        required: true
        default: "false"
        type: choice
        options:
          - "false"
          - "true"

jobs:
  build-check:
    name: Build check
    runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout repository
    uses: actions/checkout@v4      # Repository auschecken

  - name: Set up Python
    uses: actions/setup-python@v5    # fertige Action für Python
    with:
      python-version: "3.12"      # Python-Version für Backend

  - name: Install backend dependencies
    working-directory: taskmanager/app
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt

  - name: Check backend syntax
    working-directory: taskmanager/app
    run: |
      python -m py_compile app.py

  - name: Build backend Docker image
    run: |
      docker build -t taskmanager-backend:test taskmanager/app

  - name: Build frontend Docker image
    run: |
      docker build -t taskmanager-frontend:test taskmanager/web

  - name: Check Docker Compose config
    working-directory: taskmanager
    run: |
      docker compose config

```

deploy-lab:

```

name: Deploy to lab Kubernetes
needs: build-check
if: github.event_name == 'workflow_dispatch' && inputs.deploy_lab == 'true'
runs-on: self-hosted      # Runner mit Zugriff auf das Lab

```

```

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Check Kubernetes access
    run: |
      kubectl cluster-info
      kubectl get nodes

  - name: Apply Kubernetes manifests
    run: |
      kubectl apply -f k8s/taskmanager/

  - name: Wait for backend rollout
    run: |
      kubectl -n taskmanager rollout status deployment/taskmanager-backend --timeout=120s

  - name: Wait for frontend rollout
    run: |
      kubectl -n taskmanager rollout status deployment/taskmanager-web --timeout=120s

  - name: Show deployed resources
    run: |
      kubectl -n taskmanager get pods,svc,pvc

```

In der Laborumsetzung wird der CI-Teil der Pipeline automatisch ausgeführt. Er prüft, ob die Anwendung für ein späteres Deployment vorbereitet werden kann. Bei jedem push oder pull request prüft GitHub Actions die grundlegende Qualität des Projekts: ob die Python-Abhängigkeiten installiert werden können, ob der Backend-Code keine Syntaxfehler enthält, ob die Docker-Images für Frontend und Backend gebaut werden können und ob die Docker-Compose-Konfiguration für den lokalen Start gültig ist.

In einer produktionsnahen Variante kann der CI-Teil erweitert werden: zum Beispiel durch Unit Tests, Linting, Code-Quality-Checks, Security Scans der Container-Images mit Trivy sowie durch die Veröffentlichung der Images in der GitHub Container Registry (ghcr.io). Dadurch könnte die Pipeline bei kritischen Fehlern oder Schwachstellen bereits vor dem Deployment gestoppt werden.

Der Deployment-Schritt ist ebenfalls im Workflow vorgesehen. Für dieses Laborprojekt wird er aber bewusst nicht automatisch ausgeführt, weil sich der Test-Kubernetes-Cluster in einer privaten Laborumgebung befindet. Der Deployment-Job muss deshalb manuell gestartet werden und benötigt einen self-hosted GitHub Actions Runner mit Zugriff auf den privaten Kubernetes-Cluster. Dadurch bleibt die Kubernetes API privat, während die vollständige Struktur der CI/CD-Pipeline trotzdem nachvollziehbar bleibt: von der Codeprüfung bis zu einem möglichen Deployment in Kubernetes.

B. Implementiere Infrastructure as Code (IaC) für die Bereitstellung und Verwaltung der Kubernetes-Cluster-Umgebung. Wähle ein geeignetes IaC-Tool aus und begründe deine Wahl. Erstelle ein Beispiel für eine Definitionsdatei, die eine einfache Kubernetes-Cluster-Konfiguration beschreibt.

Für Infrastructure as Code verwenden wir Terraform, weil damit Infrastruktur deklarativ beschrieben, in Git gespeichert und wiederholbar erstellt werden kann. Das ist besonders wichtig für einen Kubernetes-Cluster, weil ein manuell erstellter Cluster schwieriger zu reproduzieren, zu prüfen oder fehlerfrei zu ändern ist.

In unserer Laborumsetzung existiert der Kubernetes-Cluster bereits. Er wurde separat in der Lab-Umgebung vorbereitet, deshalb erstellt dieses Repository den Cluster nicht automatisch. Im Repository wurde aber ein vereinfachtes Terraform-Beispiel für die Erstellung einer produktionsnahen Kubernetes-Umgebung in AWS ergänzt: *terraform/aws-cluster-example.tf*

Das Beispiel beschreibt einen Kubernetes-Cluster und eine Worker Node Group. Es ist bewusst vereinfacht und keine vollständige Production-Konfiguration, weil für einen realen AWS-EKS-Cluster zusätzlich IAM-Rollen, VPC, Subnetze, Security Groups und weitere cloud-spezifische Ressourcen benötigt werden.

```
# Vereinfachtes Terraform-Beispiel für einen Managed Kubernetes Cluster.
# Diese Datei ist kein vollständiges Production-Setup.
# Das Labor-Repository setzt einen vorhandenen Kubernetes-Cluster voraus.
terraform {
  required_version = ">= 1.6.0" # benötigte Terraform-Version

  required_providers {
    aws = {
      source  = "hashicorp/aws" # AWS Provider verwenden
      version = "~> 5.0"       # Provider-Version festlegen
    }
  }
}

provider "aws" {
  region = "eu-central-1" # AWS-Region Frankfurt
}

resource "aws_eks_cluster" "productline" {
  name      = "productline-cluster" # Name des Kubernetes-Clusters
  role_arn = "arn:aws:iam::123456789012:role/example-eks-cluster-role"
```

```

vpc_config {
  subnet_ids = [
    "subnet-example-private-a", # privates Subnetz A
    "subnet-example-private-b" # privates Subnetz B
  ]
}

resource "aws_eks_node_group" "workers" {
  cluster_name      = aws_eks_cluster.productline.name # Ziel-Cluster
  node_group_name   = "productline-workers"           # Name der Worker-Gruppe
  node_role_arn     = "arn:aws:iam::123456789012:role/example-eks-node-role"

  subnet_ids = [
    "subnet-example-private-a", # Subnetz für Worker Nodes
    "subnet-example-private-b"
  ]

  scaling_config {
    desired_size = 3 # gewünschte Anzahl Worker Nodes
    min_size     = 2 # minimale Anzahl
    max_size     = 5 # maximale Anzahl
  }

  instance_types = ["t3.medium"] # Beispiel-Instanztyp
}

```

Für eine reale Nutzung sollte der Terraform-State nicht lokal, sondern in einem remote backend gespeichert werden, zum Beispiel in S3 oder Terraform Cloud. Das ist wichtig für Teamarbeit, State Locking bei parallelen Änderungen und zum Schutz vor dem Verlust einer lokalen State-Datei.

C. Erläutere, wie du Configuration Management für die Konfiguration der in den Containern laufenden Microservices einsetzen würdest. Wähle ein Configuration Management-Tool aus und erkläre, wie es sich in die CI/CD-Pipeline integrieren lässt.

Als Configuration-Management-Werkzeug wählen wir hier die Kubernetes-eigenen Mechanismen, also ConfigMaps, Secrets und Umgebungsvariablen in Deployments. Für diese Aufgabe ist das sinnvoller als ein klassisches Tool wie Ansible, Puppet oder Chef, weil die Microservices in Containern laufen und die Container nicht nachträglich manuell verändert werden sollen. Das Container-Image soll möglichst unverändert bleiben, und die Konfiguration soll von außen über Kubernetes-Objekte übergeben werden. Dafür verwenden wir in unserem Beispiel ConfigMaps, Secrets und Umgebungsvariablen.

In der Laboranwendung ist die Konfiguration so aufgeteilt:

- ConfigMap: nicht geheime Konfiguration
- Secret: Demo-Zugangsdaten für PostgreSQL
- Deployment: Anbindung der Konfiguration an die Container

Zum Beispiel wird in der Datei *k8s/taskmanager/03-secret.yaml* ein Demo-Secret für die Laborumgebung gezeigt:

```

apiVersion: v1
kind: Secret
metadata:

```

```

name: taskmanager-db-secret
namespace: taskmanager
type: Opaque
stringData:
  POSTGRES_DB: taskdb
  POSTGRES_USER: taskuser
  POSTGRES_PASSWORD: taskpass

```

Das sind nur Demo-Zugangsdaten. In einer Produktivumgebung dürfen solche Werte nicht im Klartext in Git gespeichert werden. Für echte Secrets sollte man besser GitHub Secrets, während des Deployments erzeugte Kubernetes Secrets oder spezielle Werkzeuge wie Sealed Secrets, External Secrets Operator, Vault oder einen Cloud Secret Manager verwenden.

Zum Beispiel erhält der Backend-Container in der Datei *k8s/taskmanager/06-backend.yaml* die Verbindungsparameter für PostgreSQL über Umgebungsvariablen. Ein Teil der Werte wird direkt übergeben, sensible Parameter werden aus einem Kubernetes Secret gelesen:

```

env:
- name: DB_HOST
  value: taskmanager-postgres
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: taskmanager-db-secret
      key: POSTGRES_PASSWORD

```

In diesem Beispiel zeigt DB_HOST auf den internen Kubernetes Service. DB_PASS wird nicht direkt im Deployment gespeichert, sondern aus dem Secret taskmanager-db-secret gelesen. Dadurch wird die Konfiguration vom Application Code und vom Container Image getrennt.

Für das Frontend wird ebenfalls eine ConfigMap verwendet. Darin liegen *nginx.conf* und *index.html*. Für das Laborbeispiel zeigt dieser Ansatz einfach, wie Kubernetes Konfigurationsdateien an einen Container übergeben kann.

Für die Verwaltung von Konfigurationen in verschiedenen Umgebungen, zum Beispiel dev, staging und production, könnte man zusätzlich Helm oder Kustomize verwenden. Mit Helm könnte man zum Beispiel gemeinsame Templates und unterschiedliche values.yaml-Dateien für jede Umgebung nutzen. So lassen sich replicas, image tag, storage class, resource limits oder ingress host ändern, ohne alle Kubernetes-Manifeste zu duplizieren.

In die CI/CD-Pipeline lässt sich dieses Configuration Management dadurch integrieren, dass der Deployment-Job nach erfolgreichem Build die Kubernetes-Manifeste oder später die Helm-Konfiguration auf den Cluster anwendet. In der aktuellen Laborvariante ist dafür im Workflow der manuelle Deployment-Schritt mit `$ kubectl apply -f k8s/taskmanager/` vorgesehen.

Damit erfolgt das Configuration Management in unserer Lösung über Kubernetes-Manifeste: nicht geheime Konfiguration liegt in ConfigMaps, sensible Parameter liegen in Secrets, und Deployments binden diese Daten über Volumes oder Umgebungsvariablen in die Container ein. Dadurch kann die Konfiguration geändert werden, ohne sich manuell in Container einzuloggen. Das passt besser zu den Prinzipien containerisierter Anwendungen.

D. Beschreibe, wie du GitOps-Prinzipien anwenden würdest, um die Versionskontrolle und das Deployment der Infrastruktur und der Anwendungen zu verwalten. Gib ein Beispiel für einen GitOps-Workflow, der zeigt, wie Änderungen von der Entwicklung bis zur Produktion fließen.

Für die Anwendung von GitOps-Prinzipien betrachten wir Git als zentrale Quelle der Wahrheit für Infrastruktur und Application Deployment. Das bedeutet, dass Änderungen nicht manuell direkt

im Kubernetes-Cluster durchgeführt werden sollen. Stattdessen sollen alle Änderungen über Git laufen: Commit, Review, Pull Request oder Merge in den passenden Branch.

In unserem Projekt haben wir die Verzeichnisse nach ihrem Zweck getrennt:

- Application code *taskmanager/*
- Kubernetes config *k8s/taskmanager/*
- CI/CD workflow *.github/workflows/ci.yml*
- IaC example *terraform/aws-cluster-example.tf*

In einer produktionsnahen Umgebung könnte man für GitOps auch ein separates GitOps-Repository verwenden. Dort würden Kubernetes-Manifeste oder Helm Values für verschiedene Umgebungen liegen, zum Beispiel dev, staging und production.

Ein Beispiel für einen GitOps-Workflow kann so aussehen:

1. Ein Developer ändert den Application Code und erstellt einen Commit.
2. GitHub Actions startet die CI-Pipeline.
3. Die Pipeline führt Tests, Build und Security Checks aus.
4. Nach erfolgreicher Prüfung werden die Docker Images in einer Registry veröffentlicht.
5. Der Image Tag wird in den Kubernetes-Manifests oder Helm Values aktualisiert.
6. Die Änderung läuft über Pull Request und Review.
7. Nach dem Merge synchronisiert ein GitOps-Controller, zum Beispiel Argo CD oder Flux, den Kubernetes-Cluster mit dem Zustand in Git.
8. Wenn ein Deployment zurückgerollt werden muss, reicht es, den vorherigen Commit oder Image Tag wiederherzustellen.

In unserer Laborumsetzung ist der Deployment-Schritt bereits im GitHub-Actions-Workflow vorgesehen, wird aber manuell gestartet und benötigt einen self-hosted Runner mit Zugriff auf den privaten Kubernetes-Cluster. Für einen vollständigen GitOps-Ansatz könnte man als nächsten Schritt Argo CD oder Flux innerhalb des Kubernetes-Clusters installieren. Dann müsste GitHub Actions nicht direkt auf die Kubernetes API zugreifen. Die Pipeline würde nur einen neuen Image Tag veröffentlichen oder das Manifest in Git aktualisieren, und der GitOps-Controller im Cluster würde die Änderungen selbst aus dem Repository abrufen.

Dieser Ansatz hat mehrere Vorteile. Erstens werden alle Änderungen in Git versioniert und können über Reviews geprüft werden. Zweitens kann der aktuelle Zustand des Clusters mit dem gewünschten Zustand im Repository verglichen werden. Drittens wird ein Rollback einfacher, weil man zu einem früheren Commit zurückkehren kann. Außerdem muss der private Kubernetes-Cluster nicht direkt für einen GitHub-hosted Runner geöffnet werden.

Damit ergänzt GitOps die klassische CI/CD-Pipeline: CI baut und prüft die Anwendung, der CD-Prozess bereitet die Änderung der Deployment-Konfiguration vor, und der GitOps-Controller sorgt dafür, dass der Kubernetes-Cluster mit dem in Git beschriebenen Zustand synchronisiert wird.

Vielen Dank für Ihre Aufmerksamkeit.